

The stability, scalability and performance of multi-agent systems

L C Lee, H S Nwana, D T Ndumu and P De Wilde

Much has been published on the functional properties of multi-agent systems (MASs) including their co-ordination, rationality and knowledge modelling. However, an important research area which has so far received only scant attention covers the non-functional properties of MASs which include performance, scalability and stability issues — clearly these become increasingly important as the MAS field matures, and as more practical MASs become operational. An understanding of how to evaluate and assess such non-functional properties, and hence how to improve on them by altering the underlying MAS design, is gradually emerging as a pressing concern. This paper presents preliminary work to address such concerns; particularly, it investigates the performance and scalability of a multi-agent model we have developed.

Firstly, this paper defines performance, scalability and stability within the context of multi-agent applications. Following, we describe a multi-agent model that we later use to illustrate our first attempts at evolving a procedure for analysing such non-functional properties of MASs. Next, we report on our initial attempts to investigate the performance and scalability of the multi-agent model. Finally, the significance of these results in particular and of such investigations in general is discussed.

1. Introduction

Agents exist in, and only in, designated environments. In this paper, we are only concerned with rational autonomous software agents that exist in some sort of software environment [1—3]. Agents are implemented in agent programs, normally using some sort of AI system, that maps from the agent's perceptual history into its actions [4].

Multi-agent systems (MASs) are systems consisting of more than one autonomous agent that are able to interact with one another. These agents may have a global goal to solve together, or they may individually have their own goals to pursue. The particular characteristic here is that in order to achieve their goal(s), these agents must co-ordinate their actions [5]. There are many reasons for this, for example:

- to allocate resources [6—9],
- to eliminate conflicts due to their actions [10—13], or due to conflicting interests [14—19] to achieve their goal(s) which otherwise cannot be done [20, 21],
- to improve efficiency by enhancing predictability [22—25] or by reducing redundancy [20, 21],
- to improve global efficiency [26, 27].

Much recent research on multi-agent systems has therefore been mainly concerned with such functional properties as co-ordination, rationality, and knowledge modelling. Non-functional properties of the MASs, which include issues of performance, scalability and stability have so far only received scant attention. This is to some degree understandable as these issues depend upon the systems' unique underlying design and implementation technologies. Hence, they are of more concern to engineers who use these systems operationally than to scientists who carry out basic research. However, as the MAS technologies have gradually matured to be exploited in building practical distributed applications, the non-functional properties of the MASs have become increasingly important. It is now vital to research how the performance, scalability and stability of MASs affect practical design matters, and how they can be evaluated. Using such information, we could begin to understand how to improve our designs of MASs.

The breakdown of the paper is as follows. Section 2 defines performance, scalability and stability, concentrating on their meanings within the context of multi-agent systems. In Section 3, we introduce a multi-agent model that we later use for our experiments. The **preliminary** results of investigating the performance and scalability of the model are reported in Section 4. Section 5 reports on other related work. Finally, in Section 6, we conclude with a

discussion on the significance of our results in particular and of such investigations in general.

2. Performance, scalability and stability within the context of multi-agent systems

In this section, we attempt to define what performance, scalability and stability mean within the context of multi-agent systems.

2.1 Performance

Just as throughput (in terms of transactions per minute (tpm)), response time, and the number of concurrent users and tasks are important performance indicators in distributed systems [28], they are equally vital in multi-agent systems. However, there are other important factors introduced by the agent level which have costs, and therefore affect performance. They include the following.

- Co-ordination — as in co-ordination protocols employed

Co-ordination generally involves reasoning (i.e. computation costs) and communication (i.e. communication overheads). In addition, the manner in which agents are organised in a MAS has a strong impact on both the computational and communication overheads. For example, a flat organisation wherein an agent must always negotiate the terms and conditions of every service requiring joint activity with other agents is obviously more expensive computationally and communications-wise than a hierarchical society in which the behaviour of each agent engaged in a joint activity with other agents follows predefined roles.

- The agent's knowledge model

This requires sophisticated data structures (i.e. storage costs) and their manipulation (i.e. computation costs).

- The agent's rationality model

Rational reasoning entails optimisation and evaluation (computation costs).

Clearly then, as in distributed systems in general, the basic performance indicators in MASs are computational costs and throughput. Thus, performance is in fact a function of the computational complexity of the underlying implementation of the MAS. We propose the following definition for performance:

- the performance of a multi-agent system is a measure using a set of statistical indicators of the system's major outputs and its consumption of resources, where typical indicators include throughput, response time, number of concurrent agents/tasks, computational time and communications overhead.

For a multi-agent model, the variables that affect performance include, but not exclusively, the number of agents, the number of concurrent tasks/goals the agents are carrying out, the organisation of the agents and the type of co-ordination protocols employed.

2.2 Scalability

Generally, scalability refers to how well the capacity of a system to do useful work increases as the size of the system increases. For example, does the power of an engine double if the number of cylinders is doubled or can twelve agents perform twice the amount of work of six? Thus, assessing the scalability of a MAS requires that we take a macro or society view of the performance of the system at different sizes. However, from the viewpoint of a single agent in the society (i.e. a micro view), how does an increase in the size of the overall system affect that individual agent? Depending on the design and application domain of the system, such a change might either (i) not affect the agent at all, (ii) reduce the loading on the agent, or (iii) increase the loading on the agent. The first two effects (i.e. no effect or reduced loading) have little potential for degrading system performance. Further, if the contribution of each agent to the throughput of the society is additive, then these two effects lead to the society having a greater capacity for throughput when its size is increased. The third effect — increase the loading on the agent — has significant potential to degrade system performance. Thus, in distributed software engineering, the term 'scalability' is sometimes used when 'increased environmental loading due to an increase in the number of distributed components' is arguably more appropriate. For example, in an effort to demonstrate the scalability of their Windows NT solutions for mission-critical applications¹, Microsoft evaluated the system's performance (in transactions per minute) under stress (large number of concurrent users and large databases). The system's scalability was then claimed to have been enhanced with respect to a previous design, since its performance increased under the same load².

In this paper, we also use the term 'scalability' to mean increased loading on an agent which is caused by its need to interact with more agents because the size of the society has increased. Thus, the scalability of a MAS is the average measure of the degree of performance degradation of individual agents in the society as their environmental loading, caused by an expansion in the size of the society, increases. In summary then, we view the scalability of a system by taking the simplistic analytical viewpoint of algorithmic complexity [29]:

- the scalability of a multi-agent system depends on whether the worst-case performance of the system (i.e. its overall algorithmic complexity) is bounded by a polynomial function of the load.

¹ <http://www.microsoft.com/corinfo/press/1998/feb98/scalepr.htm>

² <http://www.microsoft.com/sql/guide/perf.asp>

2.3 Stability

Stability is a relatively well-understood concept in domains such as the physical sciences, where it is regarded as a property of an equilibrium. An equilibrium is stable if after a small perturbation the system returns to its equilibrium 'voluntarily'. Examples include a pendulum at rest, or an unexcited atom. For MASs, stability needs to be defined with respect to perturbations of some sort, such as noise, variations in parameter values, addition or disablement of agents, etc. Should a small disturbance at a given instant grow and become significant such that after a long time the behaviour of the system depends substantially on the initial disturbance, the system is then considered unstable.

However, the notion of an equilibrium state in MASs is not exactly clear. Intuitively, a MAS is in equilibrium when the statistical properties of its performance indicators remain stationary for a given variation in the system's external load. Thus, instability occurs when a small perturbation of some system parameter leads to a sharp and persistent deviation in the system's performance indicators.

Multi-agent systems can also be seen as agents engaged in a multi-agent game. Even in game theory, stability is a property of equilibrium. Game theory is a collection of formal models for studying decision making in strategic interactions (or conflict situations) among rational beings [30—32]. Playing a game effectively means solving the problem of choosing a rational strategy; and the outcome of the game will then be a solution for the conflict situations. Consider an electronic commerce scenario wherein different customers are engaged in (or are competing for) buying and selling different goods and services. The notion of stability is usually used to describe the characteristics of the set of strategies in equilibrium. An equilibrium point is a set of strategies from all players wherein each strategy is the best reply to others. An equilibrium point is stable if no player has any incentive to deviate from this equilibrium. For example, in Prisoner's Dilemma [33], the joint strategy 'defect-defect' is the only stable, or Nash, equilibrium [34].

3. A multi-agent model

In this section, we introduce a model of a multi-agent system, and describe a procedure to analyse its performance and scalability, thereby illustrating how we use the definitions of the previous section. In this model, a software agent is an autonomous entity with sensors for perceiving inputs and effectors for effecting outputs. It has computational abilities (i.e. reasoning, searching, etc) and can use its knowledge and rationality models to map inputs to outputs that would maximise its utility (i.e. its performance measure according to the rationality). A MAS in this paper is a set of agents organised in either a mesh or a hierarchical structure, as shown in Fig 1. In a mesh structure, every agent is connected to all other agents in the system; while agents in a hierarchical structure are

connected only to their parent and children. The organisational structure defines the communication topology of the MAS, i.e. an agent can only interact with those agents to which it is connected.

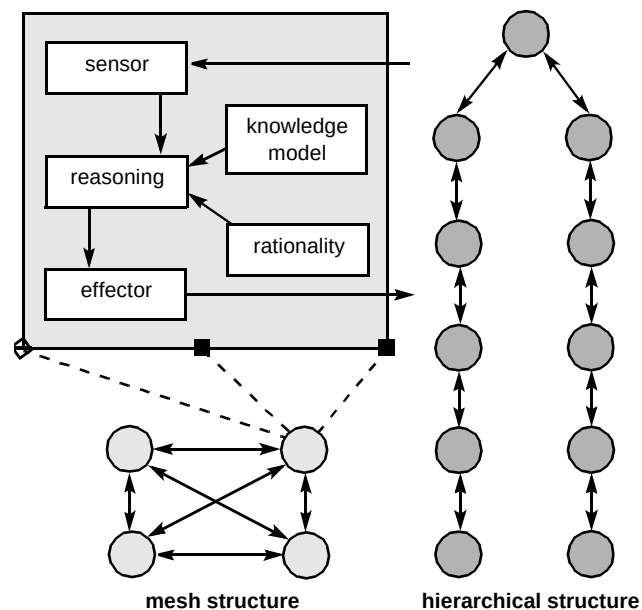


Fig 1 A multi-agent model.

3.1 Co-ordination in the multi-agent model

It is assumed that the MAS is given a task that requires all the agents to work together in order to achieve it. The task, therefore, can be imagined divisible into n sub-tasks in a MAS consisting of n agents. The MAS uses a simple contract net protocol [7] to co-ordinate the sub-task allocation problem.

Firstly, a task is given to an (server) agent by the user. This server agent divides the tasks into n' sub-tasks and then announces the sub-tasks to the n' (contractor) agents with which it can interact. The contractor agents evaluate whether they are able to perform the sub-tasks and which sub-tasks are optimal to accept. For tasks they are willing to accept, they submit a bid to the server. The server, in turn, evaluates the merits of these bids and awards the sub-tasks to the contractors with the best bid.

Should a contractor agent find that it could only perform part of a sub-task, it could in turn contract out some parts of the sub-task. Only after this agent has received bids for all the sub-sub-tasks it contracts out, is it allowed to bid for the original sub-task. Similarly, only after this agent has been awarded the contract, can it then in turn award its sub-contracts to the sub-contractors. Figure 2 describes the steps of the iterative contracting protocol used in our model.

In the model, a task is divided into n' sub-tasks by a simple database look up, i.e. the task decomposition is predefined, and the attributes associated with each sub-task

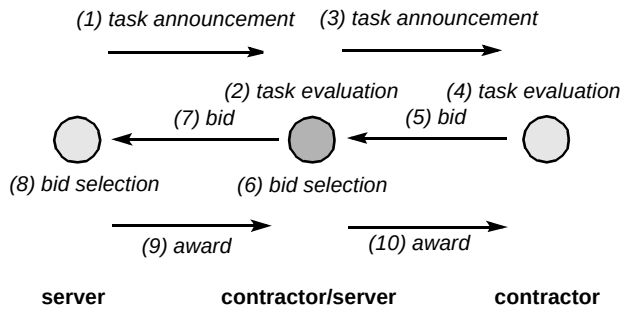


Fig 2 Iterative contracting.

are drawn randomly from a predefined set without replacement. As the number of predefined attributes is not dependent upon n' , the computational complexity of performing task decomposition can be assumed to be a constant. The bid evaluation algorithm is a utility maximisation function, and the utility of a bid is calculated using a weighted function on a finite number of attributes. Furthermore, the communication logic is a simple FIFO message queue. Since every task delegation involves a finite number of bid evaluations and messages that increase linearly with n' , its complexity is expected to be of $O(n')$. Furthermore, for m tasks, as the root agent has to perform a finite set of actions to delegate every sub-task to each child agent, the computational complexity (and hence computational time) is expected to be of $O(m)$.

3.2 Implementation

The underlying implementation of each agent abided by the following, for easy analysis:

- the use of multi-threading for each functional unit in an agent — functional units here include incoming communications (sensors), outgoing communications (effectors), and the co-ordination engine (reasoner),
- the use of asynchronous communication,
- the databases for the agents' knowledge model are distributed but not shared.

In other words, our MAS model resembles a network of individual processes with their own logics. Each process has its own resources and reasoning logic that allow it to achieve certain tasks. Processes interact with one another by asynchronous communication only (i.e. message passing) in a loosely coupled manner, requiring agents to solicit the help of others in order to achieve their goals. As a process/agent is autonomous, it will only help if its rationality model deems it necessary to do so.

Multi-agent systems based on the above model were implemented using the ZEUS tool-kit [35] in Java JDK 1.1 and run on Sun Unix workstations under Solaris 2.5.1. Each agent's knowledge model defined the contract net protocol and task decomposition while its rationality model allowed bids to be evaluated. Also, each agent's reasoner co-

ordinates the task allocation with other agents according to its predefined knowledge and rationality model. The sensors and effectors deal with agent communication, and the implementation uses the KQML Agent Communication Language [36] and the TCP/IP protocol for transport.

3.3 Performance and scalability of the implemented model

In this section, we describe how we evaluated the performance and scalability of an MAS implementation based on the above model. It is noted that the model *per se* (i.e. its algorithms) is scalable on the basis of our earlier analysis where the computational complexity was shown to be a polynomial function of the load. The complexity of the implementation itself, however, which includes the realisation of the model and some additional functionalities (e.g. the scheduling of concurrent tasks, sub-tasks delegation, concurrent multi-agent co-ordination and concurrent messaging), has become much less clear. It is for this reason that the implementation of the MAS needs to be analysed, even if the complexity of the MAS model is well understood.

The analysis is confined to computational time, as the system's major performance indicator, and number of tasks and agents, as its typical load. Drawing from the definitions given earlier, the scalability of the system is a test of whether its worst-case computational time is bounded by a polynomial function of the number of tasks and agents. We also vary the organisation of the agents. This leads us to evaluate two organisations of the agents — a mesh organisation and a hierarchically structured organisation.

Firstly, the performance of the mesh-structured MAS was evaluated through simulations. In each simulation, a set of m tasks was given to the MAS comprising n agents. Each task can be divided into n sub-tasks and can only be achieved if every one of the n agents works on a sub-task. Agents co-ordinate the allocation of these sub-tasks using the contract net co-ordination protocol. The total computational time of the root agent to achieve all the tasks was then measured. To analyse the system's scalability, the simulation was repeated using increasing numbers of tasks and agents.

Secondly, we repeated the simulations with the hierarchically structured MAS. These hierarchical MASs have tree-like structures where the root agent has one or more child agents; the terminal leaf agent has no child while each of the remaining agents has one child only. An agent can interact with either its children or parent. The number of children that the root agent has is determined by the depth of the tree and the total number of agents. For example, in a 5-level hierarchical MAS (i.e. the tree has 5 levels of nodes), the root agent has one, three, and ten child agents respectively if the MAS has 6, 16, and 101 agents in total (see Fig 1). We simulated two hierarchical organisations — a 5-level and 10-level hierarchical MAS. Here, every task is

divided into $n' + 1$ sub-tasks where n' is the number of child agents of the root. Every sub-task at each level is further divided into two — one for the agent at this hierarchy and one for its child agent at a lower hierarchy — and the same division repeats until the bottom level. Similarly, we calculated the total computational time of the root agent to achieve all the tasks. In summary, the following data from the simulations has been collected, analysed and evaluated:

- for the mesh-structure MASs:
 - the computational time of the system as the number of tasks was increased,
 - the computational time of the system as the number of agents was increased,
 - the boundary conditions beyond which the system became unstable or broke down,
- for the hierarchical-structured MAS:
 - the computational time of the system as the number of agents was increased.

4. Simulations results

4.1 Mesh-structure MASs

Figure 3 shows the relationships between computational time and the number of tasks and agents. It can be seen that our expectations are basically valid. Since the degradation in the system performance is a polynomial function of the load (i.e. the number of tasks and agents), the system scales computationally. However, as the number of tasks and agents both exceeds 100, the system's performance degradation begins to grow exponentially with load — that is, it becomes non-scalable. The simulations broke down at these points because the memory required by each agent exceeds the Java virtual machine limit.

Strictly speaking, the system is not non-scalable above loads of more than 100 agents; rather, the underlying implementation technology³ does not scale. Had the system been implemented using different technologies (e.g. C++ or had we employed a native Java compiler), the system would have scaled beyond the limits shown. No matter what underlying technologies are used, however, there will

always be a physical or logical limit beyond which the system no longer scales. Figure 4 shows how the average computation time per tasks is related to the number of agents in the MAS. This summary provides valuable

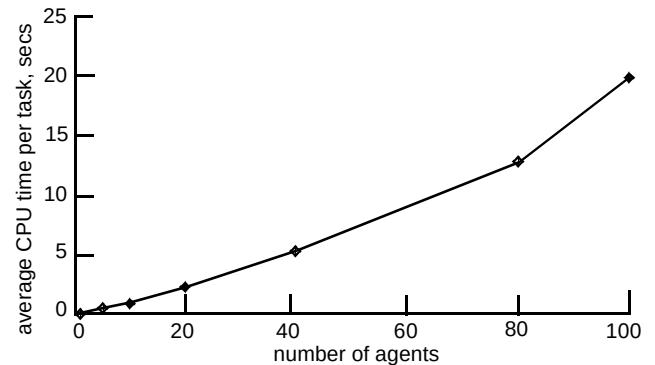


Fig 4 Average computational time per task versus number of agents in mesh structures.

information to estimate the average throughput and response time of MASs when given the system's typical load. Our simulations have ascertained the limits at which the system starts becoming unpredictable and no longer scales. It is clearly useful to establish the boundary conditions within which to operate the specific implementation of the system so that a guaranteed quality of service (QoS) can be ensured. Since the bottle-neck of the system is memory, increasing the number of concurrent agents in the system will consume more memory. Therefore, the number of concurrent tasks possible would be reduced — and vice versa. The maximum number of tasks is therefore expected to be inversely proportional to the number of agents:

$$m_{max} = C/n_{max}$$

where m_{max} and n_{max} are the maximum number of tasks and agents respectively, and C is a constant. Figure 5 shows that the experimental boundary conditions approximate the predicted curve⁴ quite closely.

³ In this case, the Java virtual machine

⁴ C is approximated by computing the sizes of agent and task objects, and by assuming the maximum memory size of the Java virtual machine to be 20 Mbytes.

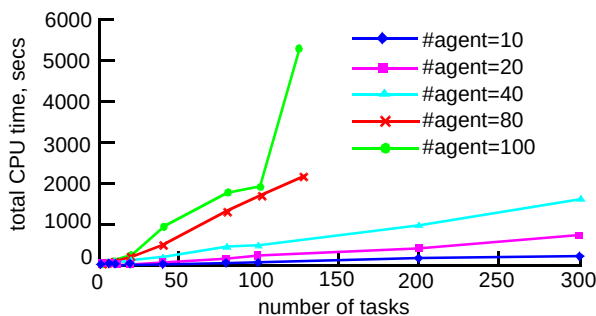
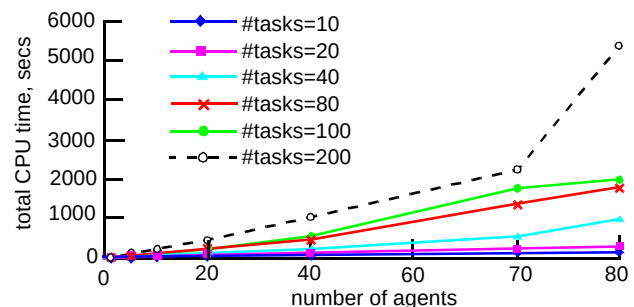


Fig 3 Total computation time versus number of tasks and agents in mesh structures.



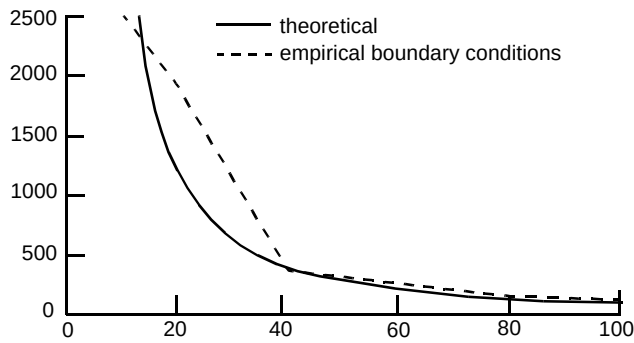


Fig 6 The boundary conditions.

4.2 Hierarchical-structure MASs

In organisational theories, one of the main advantages of using various organisational structures is to cope with the increased communication caused by uncertainty [37]. In our model, the hierarchical structure reduces the root agent communication and hence computational complexity by limiting the number of its child agents and by delegating the task allocation responsibility to other agents.

As explained previously, in the 5-level hierarchical system, the number of agents with which the root agent interacts is reduced fivefold as compared to the mesh structure; similarly, it will be reduced tenfold in the 10-level system. The amount of communication, and hence computation, involved will also be reduced by the same degree. Moreover, the root agent will only be required to decompose tasks into n' sub-tasks where $n' \leq n$, which is therefore computationally less expensive. As a result, the computational time of the root agent in the 5-level hierarchical structure is expected to be much less than that in the mesh structure; and the time required in the 10-level hierarchical structure is similarly expected to be less than that in the 5-level hierarchical structure. Figure 6 shows that this is exactly the case as expected..

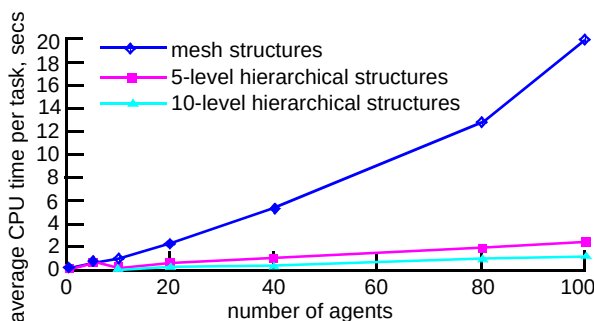


Fig 6 Average computational time per task versus number of agents in mesh and hierarchical structures.

In addition to the contract net protocol, we repeated the simulations using an auction protocol. The auction protocol

is like a repetitive contract net, where the server agent, instead of awarding a bid from all the bids that it has received, modifies its original task based on the received bids and re-announces the task. For example, the server agent could substitute the price of the task with the lowest bidding price received and re-announce the tasks. The protocol will end by awarding a contractor with the most optimal bid when the server cannot find any more optimal bid, i.e. the bidding process has converged.

Since each round of the bidding process involves communication (and bid evaluation), it is computationally expensive. Therefore it is expected that the computational cost (to the root agent) using the auction protocol in a mesh structure is greater than is the case when using the contract net protocol. In Fig 7, the empirical result is found to be consistent with our expectation.

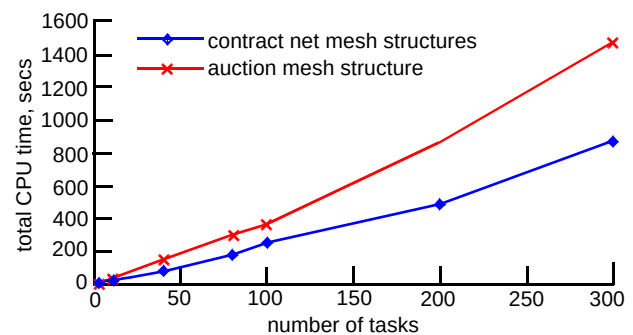


Fig 7 Total computational time versus number of tasks using different protocols.

It is worth noting that the experiments on the auction protocol have implicitly assumed that the protocol will converge in finite time. It is, however, not always the case in practice. Had a multi-agent system been designed without a guaranteed convergent co-ordination protocol, the system could not have been stable and scalable. Whether a protocol is convergent is partly determined by the sequence in which proposals (i.e. task requests and bids) are announced, and the evaluation functions that agents use to select (or not select), and the response to the proposals. The sufficient conditions for a convergent co-ordination protocol in different multi-agent models have been widely investigated [14, 19, 38–41].

Furthermore, in order to find a 'correct' sequence for these proposals to be announced, agents may be required to enumerate a substantial number, if not all, of the proposals before the co-ordination can commence. Obviously this is an extremely expensive process and may not be viable because of agents' bounded rationality [42]. We have proposed a convergent protocol for this problem that requires agents only to produce one proposal at a time [43].

5. Related work

There have been very few papers in the literature which cover such non-functional issues pertaining to the development of distributed agent applications. One of the reasons for this is probably because not many distributed agent applications are available which would necessitate such investigations. Furthermore, the implementation-specific nature of such investigations clearly suggests that their results are not generalisable. This is why in this paper, we have concentrated to a large degree on two issues:

- clarifying what performance, scalability and stability mean in the context of multi-agent systems [44],
- demonstrating how the performance and scalability of a MAS could be systematically analysed [33].

We also summarise below among other things why such investigations are necessary.

There are indeed some similar works on this subject. Lejter and Dean [45] describe a software system that supports distributed computing and multi-agent co-ordination. Using the system, they have been able to experiment with various multi-agent co-ordination protocols. They report, as we have done in this paper, on scalability and performance experiments in which they vary the organisation of the agents and the protocols employed.

Moreover, performance and scalability experiments on distributed client/server systems have been reported [46]. The scalability of the client/server system is measured in terms of response time under load (number of tasks). The client/server system has a layered structure where servers are organised in a multi-level hierarchy, which is similar to our hierarchical structures.

Performances of various different communication software systems and environments (CORBA, TVM and BSD under Ethernet, FDDI, HiPPi and ATM) for distributed systems under various conditions (e.g. message size) are evaluated [47]. While our work concentrates on evaluating the performance of different task allocation schemes, their results analyse the overheads of various communication means for allocating tasks.

In other work, we have also investigated the stability of a multi-agent system modelled by a stochastic process [48, 49]. The system is based on resource re-allocation through a memoryless bidding protocol where every agent calculates its next bidding price using a discount drawn from a Gaussian distribution. Our simulation shows that the

network of stochastic agents reaches an equilibrium distribution, even in the presence of noise.

6. Conclusions

In this paper we have reported on non-functional issues which pertain to distributed agent systems including performance, scalability and stability. The contributions of this paper are the following:

- highlighting the fact that research into such non-functional issues is scant, and exploring what performance, scalability and stability mean in the context of multi-agent systems,
- demonstrating how to analyse the performance and scalability of multi-agent systems,
- highlighting the fact that the operating boundaries of such systems (in our case the maximum number of tasks and agents that the system can sustain) need to be investigated for deployed applications.

The benefits of carrying out such investigations become increasingly important as MASs mature, and this paper has shown that different MAS designs need to be investigated in order to optimise these non-functional requirements. More importantly, such requirements clearly underpin the quality of service that will need to be guaranteed to users as multi-agent systems become available.

In this vein, the sort of investigations reported in this paper will be of paramount importance. We hope the reader agrees that though the paper is not comprehensive, it does make a contribution in highlighting the fact that non-functional issues in agent systems need to be investigated thoroughly. This paper is a start down this road.

Acknowledgements

We acknowledge the comments of the three reviewers of this paper.

We are also indebted to Alan Steventon and Nader Azarmi whose reviews and views have shaped the contents of this paper.

References

- 1 Nwana H S: 'Software agents: an overview', *Knowledge Engineering Review*, 11, No 3 (1996).
- 2 Bradshaw J: 'Software agents', MIT Press (1997).

- 3 Huhns M H and Singh M P: 'Readings in agents', Morgan Kaufman (1998).
- 4 Russell S and Norvig P: 'Artificial intelligence: a modern approach', Prentice Hall (1995).
- 5 Nwana H S, Lee L C and Jennings N: 'Co-ordination in software agent systems', *BT Technol J*, 14, No 4, pp 79—88 (1996).
- 6 Gasser L and Bond A H: 'Reading in distributed artificial intelligence', Morgan Kaufmann (1988).
- 7 Davis R and Smith R G: 'Negotiation as a metaphor for distributed problem solving', *Artificial Intelligence*, 20, No 1, pp 63—109 (1983).
- 8 Malone T W, Fikes R E, Frant K R and Howard M T: 'Market-like load sharing in distributed computing environments', Technical Report Center for Information Systems Research Sloan School of Management, Working Paper No 139, Massachusetts Institute of Technology (1987).
- 9 Sathi A and Fox M S: 'Constraint-directed negotiation of resource reallocation', in Huhns M and Gasser L (Eds): 'Distributed artificial intelligence', 2, chapter 8, pp 163—193, Pitman (1989).
- 10 Decker K and Lesser V: 'An approach to analyzing the need for meta-level communication', Proceedings of the Thirteenth International Joint Conference on Artificial Intelligence, Chambéry, France (1993).
- 11 Georgeff M: 'Communication and interaction in multi-agent planning', Proceedings of the 1983 National Conference Artificial Intelligence, pp 125—129 (August 1983).
- 12 Steeb R, Cammarata S, Hayes-Roth F A, Thorndyke P W and Wesson R B: 'Architectures for distributed intelligence for air fleet control', Technical report, Rand Co-operation (1981).
- 13 Zlotkin G and Rosenschein J S: 'One, two, many: coalitions in multi-agent systems', Proceedings of the Fifth European Workshop on Modelling Autonomous Agents in Multi-Agent World, Neuchatel, Switzerland (August 1993).
- 14 Lee L C: 'Progressive multi-agent negotiation', Proceedings of the Second International Conference on Multiagent Systems, pp 173—180, Kyoto, Japan (December 1996).
- 15 Conry S E, Kuwabara K, Lesser V R and Meyer R: 'Multistage negotiation for distributed constraint satisfaction', *IEEE Trans SMC*, 21, No 6, pp 1462—1477 (1991).
- 16 Sycara K P: 'Resolving goal conflicts via negotiation', Proceedings of the Seventh National Conference on Artificial Intelligence, pp 245—250 (1988).
- 17 Rosenschein J S and Genesereth M R: 'Deals among rational agents', Proceedings of the Ninth International Joint Conference on Artificial Intelligence, pp 91—99 (1985).
- 18 Werkman K J: 'Multiagent co-operative problem solving through negotiation and sharing of perspectives', PhD thesis, Lehigh University (1990).
- 19 Zlotkin G and Rosenschein J S: 'Negotiation and task sharing among autonomous agents in co-operative domains', Proceedings of the Eleventh International Joint Conference on Artificial Intelligence, pp 912—917 (1989).
- 20 Durfee E H, Lesser V R and Corkill D D: 'Coherent cooperation among communicating problem solvers', *IEEE Trans Comput*, 11, pp 1275—1291 (1987).
- 21 Lesser V R and Corkill D D: 'The distributed vehicle monitoring testbed: a tool for investigating distributed problem solving networks', *AI Magazine*, pp 15—33 (1983).
- 22 Durfee E H, Lesser V R and Corkill D D: 'Trends in cooperative distributed problem solving', *IEEE Knowledge and Data Engineering*, 1, No 1, pp 63—83 (1989).
- 23 Durfee E H and Lesser V R: 'Using partial global plans to coordinate distributed problem solvers', Proceedings of the Tenth International Joint Conference on Artificial Intelligence, pp 875—883 (1987).
- 24 Durfee E H: 'Co-ordination of distributed problem solvers', Kluwer Academic Publishers (1988).
- 25 Gmytrasiewicz P J and Durfee E H: 'A rigorous, operational formalization of recursive modeling', Proceedings of the First International Conference on Multiagent Systems, San Fransisco, USA (June 1995).
- 26 Briggs W and Cook D: 'Flexible social laws', Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence, pp 688—693, Montreal, Canada (August 1995).
- 27 Shoham Y and Tennenholtz M: 'On social laws for artificial agent societies: off-line design', *Artificial Intelligence*, 73, Nos 1—2, pp 231—252 (February 1995).
- 28 Coulouris G, Dollimore J and Kindberg T: 'Distributed systems: concepts and design', Addison-Wesley (1994).
- 29 Cormen T H, Leiserson C E and Rivest R L: 'Introduction to algorithms', MIT Press (1990).
- 30 Binmore K: 'Fun and games: a text on game theory', D C Heath (1992).

- 31 Luce R D and Raiffa H: 'Games and decisions', John Wiley & Sons Inc (1957).
- 32 Myerson R B: 'Game theory, analysis of conflict', Harvard University Press (1991).
- 33 Axelrod R: 'The Evolution of Co-operation', Basic Book (1984).
- 34 Nash J F: 'The bargaining problem', *Econometrica*, 18, pp 155-162 (1950).
- 35 Nwana H S, Ndumu D T and Lee L C Z: 'A collaborative agents toolkit', *Proceedings of the Third International Conference on the Practical Application of Intelligent Agents and Multi-Agent Technology*, pp 377-392 (1998).
- 36 Finin T, Labrou Y and Mayfield J: 'KQML as an agent communication language', in Bradshaw J (Ed): 'Software agents', MIT Press (1997).
- 37 Galbraith J R: 'Organization design', Addison-Wesley (1977).
- 38 Lee L C: 'Negotiation strategies and their effect in a model of multiagent negotiation', PhD thesis, Department of Computer Science, University of Essex, Colchester, UK (July 1996).
- 39 De Wilde P: 'Nonlinear dynamics of two types of networks with intelligent nodes', *BT Technol J*, 14, No 4, pp 98-104 (1996).
- 40 Rosenschein J S and Zlotkin G: 'Rules of encounter: designing conventions for automated negotiation among computers', MIT Press (1994).
- 41 Zeuthen F: 'Problems of Monopoly and Economic Warfare', G Routledge & Sons (1930).
- 42 Simon H: 'The Sciences of Artificial', Pitman (1981).
- 43 Lee L C: 'A model of progressive multi-agent negotiation', *Proceedings of the Third International Conference on Multiagent systems*, Paris, France (July 1998).
- 44 Collins Cobuild English Language Dictionary (1989).
- 45 Lejter M and Dean T: 'A framework for the development of multiagent architectures', *IEEE Expert Intelligent Systems and their Applications*, 11, No 6, pp 47-59 (1996).
- 46 Woodside C M and Schramm C: 'Scalability and performance experiments using synthetic distributed server systems', *Distributed Systems Engineering*, 3, Nos 2-8 (1996).
- 47 Fatoohi R A: 'Performance evaluation of communication software systems for distributed computing', *Distributed Systems Engineering*, 4, pp 169-175 (1997).
- 48 De Wilde P, Nwana H S and Lee L C: 'Robust stable strategies for networks of intelligent personal assistants', submitted to *Distributed Systems Engineering* (1998).
- 49 De Wilde P, Nwana H S and Lee L C: 'Stability, fairness and scalability of multiagent systems', submitted to *Journal of Parallel and Distributed Computing* (1998).



Lyndon Lee received an MSc in Computer Science (Artificial Intelligence) with Distinction and a PhD in Computer Science from the University of Essex in 1989 and 1996 respectively. His doctoral thesis was in a formal model of multi-agent negotiation. From 1992-5 he was a recipient of the Croucher Foundation scholarship, and between 1987 and 1992 he worked as a developer and a consultant with a number of companies. After a short-term research fellowship with BT Laboratories (BTL), he joined the Intelligent Systems Research group in BTL in 1996, carrying out research and development of real-world multi-agent systems. In 1997, the ZEUS project that he co-developed in BT won the BCS IT award and since then has four patents on application. His research interests lie primarily in distributed artificial intelligence, artificial life, decision theory, game theory, and distributed and nonlinear systems. He is a member of BCS.



Hyacinth S Nwana is a principal research scientist/engineer and technical group leader in the Applied Research and Technology (ART) department at BT Laboratories. He holds a BSc in Computer Science and Electronic Engineering, an MSc in Computer Science and a PhD in Artificial Intelligence/Computer Science (1988). He also recently completed (November 1997) an MEd degree in Computer Science Education at Queen's College, the University of Cambridge.

Between 1989 and 1995, he worked in various roles (lecturer, researcher, visiting or contract scientist) for the Universities of Liverpool, Keele and Calgary (sponsored by the Royal Society), Shell Research Labs, Unilever Research Labs and BT Laboratories (BTL). He joined BTL fully in October 1995.

In 1991, he principally won the DEC European AI prize. In 1997, he led BT's agents-based project (ABW-ZEUS) which won the prestigious British Computer Society top award for innovation. He is a member of the British Computer Society and a Chartered Engineer.

He currently runs ART's Future Technologies group investigating novel biologically motivated computing models, software agents, believable interface agents, cognitive systems, and the application of such techniques to telecommunications and other computing problems.

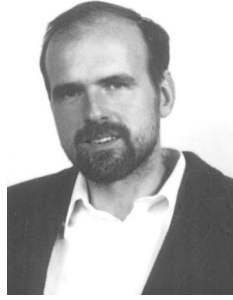


Divine Ndumu graduated in Civil Engineering from Imperial College in 1989, and Mathematics and Computing from the Open University in 1991. In 1989, he was winner of the Imperial College Governor's prize for excellence in Civil Engineering. In 1993 he was awarded a PhD in Engineering Applications of Artificial Intelligence from Imperial College.

In 1992, he joined the University of Central Lancashire as a Lecturer, and in 1996/97 served a one year Research Fellowship secondment at BT Laboratories (BTL). In

April 1997, he joined the Intelligent Systems Research Group at BTL, and currently leads the group's software agent research programme. In 1997, the ZEUS agent building tool-kit that he co-developed won the prestigious BCS IT award.

The focus of his current research is on the practical applications of AI techniques to real-world problems.



Philippe De Wilde obtained the PhD in Mathematics and MSc in computer Science from the Universiteit Gent, Belgium, in 1985. He has been a lecturer in the Department of Electrical and Electronic Engineering of Imperial College of Science, Technology and Medicine since 1989. He has worked on the connectivity of nonlinear dynamical systems, especially neural networks. He is the author of a textbook on neural networks.

He has been collaborating since 1994 with the Intelligent Systems Research group at BT Laboratories. His focus within this col-

laboration is on networks with intelligent nodes, intelligent agents, and soft agents.